Object Oriented Programming (CSEG1044)
Unit 06: Capstone Project — Phase 2

UML and Package Design

Project: Cafeteria Order and Billing System

1. Goal of This Phase

This phase converts the use cases from Phase 1 into a concrete class design and package structure. Every class is given a clear responsibility, relationships between classes are defined, and the architecture is split into four distinct layers: model, dao, service, and ui.

2. Package Structure

The project follows a four-layer architecture. Each package has a single, well-defined role. SQL never appears in UI code, and bill calculation logic never appears inside Swing event handlers.

Package	Layer	Responsibility
model	Data Layer	Plain Java classes (POJOs) representing real-world entities. No SQL, no Swing.
dao	Persistence Layer	All JDBC/SQL code lives here. One DAO class per entity. Returns model objects to callers.
service	Business Logic Layer	Validation, calculations, and workflow coordination. Calls DAO, returns results to UI.
ui	Presentation Layer	Swing screens only. Calls service layer. Contains zero SQL and zero business logic.
util	Utility Layer	Singleton DBConnection class manages the JDBC connection used by all DAO classes.

3. Folder Layout

The recommended folder layout inside the src directory is:

```
src/
  cafeteria/
    model/
      User.java
      MenuItem.java
      Customer.java
      Order.java
      OrderItem.java
      Bill.java
```

```
dao/
  UserDao.java
  MenuDao.java
  OrderDao.java
service/
  BillingService.java
  MenuService.java
  OrderService.java
ui/
  LoginFrame.java
  AdminDashboard.java
  OperatorDashboard.java
  MenuManagementPanel.java
  PlaceOrderPanel.java
  BillPanel.java
  OrderHistoryPanel.java
  SalesSummaryPanel.java
util/
  DBConnection.java
Main.java
```

4. Class Responsibilities

4.1 Model Classes (package: cafeteria.model)

Class	Responsibility
User	Stores login credentials and role (Admin/Operator). Provides validatePassword() so password checking stays out of the UI.
MenuItem	Holds item name, category, price, and availability flag. No calculation logic.
Customer	Stores customer name and contact information linked to an order.
Order	Holds a list of OrderItem objects and a reference to Customer. Provides getTotal() to sum all line totals.
OrderItem	Joins a MenuItem to an Order with a quantity. Provides getLineTotal() = unitPrice × quantity.
Bill	Holds the computed subtotal, tax, and final total for one Order. Does NOT calculate — values are set by BillingService.

4.2 DAO Classes (package: cafeteria.dao)

Class	Responsibility
UserDao	Queries the users table. Provides findByUsername() used during login. All SQL is here, not in LoginFrame.

MenuDao	Full CRUD for the menu_items table. Validates uniqueness of item names before insert.
OrderDao	Persists orders, order_items, and bills to the database. Provides getByDate() for daily sales queries.

4.3 Service Classes (package: cafeteria.service)

Class	Responsibility
BillingService	Calculates subtotal, tax (5% GST), and total. Creates and returns a Bill object. Tax logic is never inside a Swing listener.
MenuService	Validates menu item fields before calling MenuDao. Ensures name uniqueness. Acts as gatekeeper between UI and DAO.
OrderService	Validates and places orders via OrderDao. Retrieves order history and computes daily sales summaries.

4.4 Utility Class (package: cafeteria.util)

Class	Responsibility
DBConnection	Singleton class that creates and returns a single shared JDBC Connection. All DAO classes call DBConnection.getInstance().getConnection() — never create their own Connection.

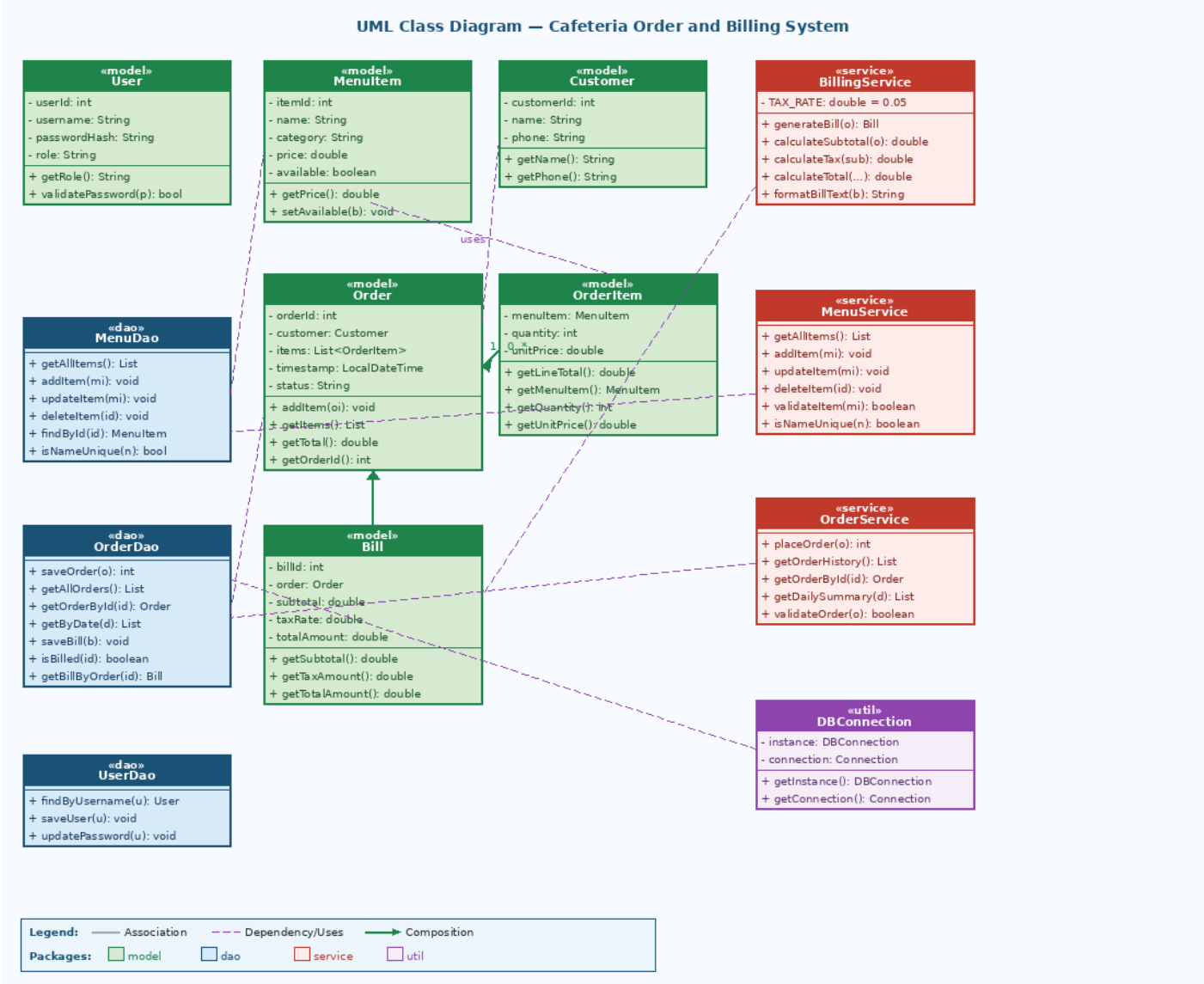
5. Key Relationships

From	To	Type	Meaning
Order	**OrderItem**	Composition (1 to 0..*)	An Order owns a list of OrderItems. Items cannot exist without an Order.
OrderItem	**MenuItem**	Association	Each OrderItem references one MenuItem for name and price.
Order	**Customer**	Association	Each Order is linked to one Customer.
Bill	**Order**	Composition	A Bill is generated from one Order and holds its calculated totals.
MenuService	**MenuDao**	Dependency	Service calls DAO to read/write menu items from the database.

OrderService	**OrderDao**	Dependency	Service calls DAO to persist and retrieve orders and bills.
BillingService	**Bill**	Dependency	Service creates and populates a Bill object after calculation.
All DAOs	**DBConnection**	Dependency	Every DAO gets the shared JDBC Connection from DBConnection.

6. UML Class Diagram

The diagram below shows all classes grouped by package, with fields, methods, and relationships. Green = model, Blue = dao, Red = service, Purple = util.



7. Design Warnings Addressed

Warning from Phase 2 PDF	How This Design Addresses It
Do not create one giant manager class	Responsibilities are split into BillingService, MenuService, and OrderService — no single God class.
Do not place SQL inside Swing event handlers	All SQL is in DAO classes. UI panels only call service methods.
Do not keep validation logic only in the UI	Validation is in service classes (e.g., MenuService.validateItem). UI may show errors but never decides rules.
Do not create classes with no real responsibility	Every class in this design has a clearly defined, unique role as shown in Section 4.

8. Review Checklist

	Checklist Item
✓	Every class has a clear, single role — no class does everything
✓	Relationships (association, composition, dependency) are logically correct
✓	Service and DAO responsibilities are separated — no SQL in service or UI classes
✓	The UML diagram supports all 7 use cases from Phase 1
✓	Bill calculation is in BillingService, not inside a Swing listener
✓	DBConnection is a Singleton — only one connection is created
✓	All model classes are plain Java objects with no SQL or Swing imports